Why PostgreSQL?

An interview answer: What is PostgreSQL, and why use it instead of MySQL or MongoDB — with real examples

Built around the Nodebase AI-workflow platform (Next.js + Prisma/Postgres) • PostgreSQL 16 / MySQL 8 / MongoDB 7 • Generated 2026

ACID + MVCC | JSONB Hybrid Store | FK + CHECK + Exclusion | pgvector / pg_cron | Prisma First-Class

1. The Answer in One Breath

"PostgreSQL is an open-source, ACID-compliant object-relational database. We chose it for an AI-workflow product because our core domain is relational by nature — nodes connected by edges, executions referencing workflows, credentials referenced by nodes — so we need joins, foreign keys and *multi-row transactions* that hold unconditionally. PostgreSQL gives us that first-class integrity via constraints and MVCC, plus *JSONB* for the flexible, document-shaped configuration data that would otherwise push us toward MongoDB, and extensions such as pgvector for the AI pieces, all in one database."

If you only have 30 seconds

PostgreSQL is a standards-compliant relational DB with strong ACID guarantees and a huge extension ecosystem. We used it because: (1) our data model is genuinely relational (graph + execution lineage), (2) we need atomic multi-table writes and hard constraints, and (3) JSONB + extensions give us the document flexibility and AI features without introducing a second database.

2. What Is PostgreSQL?

PostgreSQL (started in 1986 at Berkeley as *POSTGRES*, released to the community in 1996) is an advanced open-source object-relational database management system (ORDBMS). It speaks SQL, is ACID-compliant, and implements MVCC (multiple versions of each row) so readers never block writers. Unlike a document store, it enforces schema and integrity; unlike a plain relational DB, it is deeply *extensible*: you can define custom types, functions, operators, and even new index methods, and you can store JSON without losing relational guarantees.

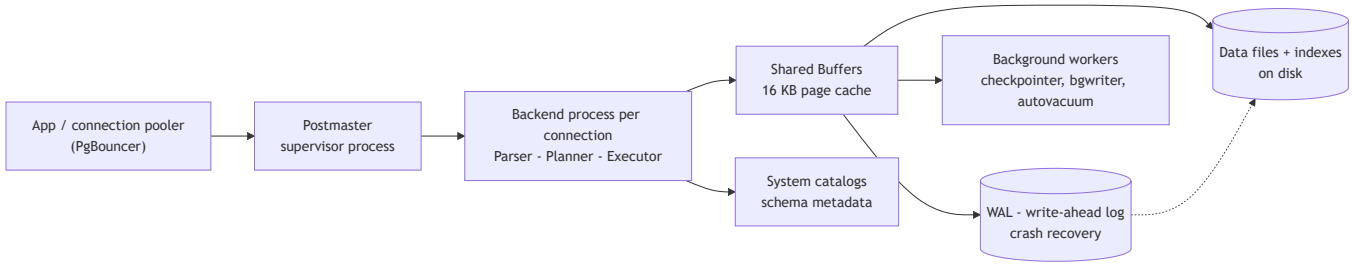


Figure 1 — PostgreSQL internals: postmaster → per-connection backends → shared buffers → WAL + data files

Capabilities that matter

Capability	What it means
ACID + MVCC	Atomic, consistent, isolated, durable multi-row transactions with safe concurrent reads/writes.
Full constraint model	PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, EXCLUDE, deferrable constraints — enforced, not advisory.
Advanced SQL	CTEs (incl. recursive), window functions, `RETURNING`, `LATERAL`, upsert, partial/materialized views.
JSONB	Binary JSON with full relational coexistence — GIN-indexable, queryable with `@>`/`jsonb_path_query`.
Indexing	B-tree, GIN, GiST, SP-GiST, BRIN, hash, expression/partial indexes; online `CREATE INDEX CONCURRENTLY`.
Extensions	pgvector (embeddings), pg_cron (scheduling), PostGIS (geo), uuid-ossp, pg_trgm (fuzzy text) — community packages that plug into the engine.
Operational	WAL crash recovery, point-in-time recovery, streaming replication, logical replication, partitioning.

3. PostgreSQL vs MySQL

MySQL is the most popular web database; PostgreSQL is the more standards-complete and feature-driven one. Modern MySQL 8 closed much of the gap (CTEs, window functions, CHECK, JSON cells), so the honest answer is "it depends" — but for our workload PostgreSQL still wins on integrity and advanced features.

Dimension	PostgreSQL	MySQL (InnoDB)
SQL standard	Closest-to-standard; richest feature-first adoption	**Fluent but historically non-standard quirks; CATALOG/names-case sensitivity**
Data integrity	Strict types & full constraint set; no silent truncation / zero-dates permissive mode	**Permission-reduced "sql_mode"; truncation & implicit casts historically allowed**
JSON storage	JSONB: binary, indexable with GIN, coexists with relational columns	**JSON type (text-based, uses `JSON_TABLE` for joins)**
Indexes	GIN/GiST/BRIN/partial/expression; `CREATE INDEX CONCURRENTLY`	**B-tree + R-tree; online DDL improvements in 8.x but more limited**
Concurrency	MVCC; no table locks on most DDL; better read/write mixes under contention	**MVCC too; table/catalog locks historically worse, improved in 8.0**
Extensions	pgvector, pg_cron, PostGIS, pg_repack, ...	**Fewer; partitioned setup**
Replication	**Streaming/logical replication, powerful but more moving parts**	Native async/sync replication, GTID, widely operated; Vitess for sharding
Operations	**More config surface, autovacuum needs tuning**	Smaller footprint, huge DBA talent pool, simple tune

When MySQL is the right answer — be honest

- Simple **read-heavy CRUD** web apps where the queries are straightforward.
- Existing **team expertise and operations** are MySQL-shaped; you need very wide scale-out via Vitess.
- The team wants the smallest operational surface (fewer concepts: vacuum, WAL, roles, etc.).
- Say: "In a pure read-mostly content service I would not fight for Postgres. The differentiator here is the transactional graph engine."

4. PostgreSQL vs MongoDB

MongoDB is a document database: collections of JSON-like documents with dynamic schemas, replicated and sharded for horizontal scale. The sales pitch is "flexible schema + scale-out". PostgreSQL answers both of those with JSONB + GIN and by scaling vertically (and horizontally with Citus/partitions) while *keeping* joins, transactions and constraints — which MongoDB does not have as first-class citizens.

Dimension	PostgreSQL	MongoDB
Data model	Relational + JSONB hybrid	Schemaless documents; relationships are manual (embedded/refs)
Referential integrity	Foreign keys enforced by the engine	No FKs — dangling refs are silently possible
Multi-record transactions	Full ACID multi-table transactions, serializable isolation	Multi-document transactions only on replica sets; historically limited
Consistency	Strongly consistent reads by default	Tunable read/write concern; eventual by default on secondaries
Constraints	UNIQUE / CHECK / EXCLUDE / deferrable	Unique indexes only (no CHECK/exclusion), partial only
Joins / analytics	Full JOIN + window functions + CTEs	$lookup pipelines; aggregation framework; heavier for graph-shaped data
Scale-out	Vertical; partitioning; Citus for distributed	Native sharding, write-anywhere horizontal scale

Why a workflow engine is a poor fit for MongoDB

- A workflow is a **graph**: `node` and `connection` rows reference each other; the execution engine topological-sorts those edges. That's joins + FKs.
- Saving a workflow means **replacing the whole graph atomically** — a multi-collection transaction where correctness (no half-saved graph) is non-negotiable.
- Executions form a **lineage** (workflow → many runs, each with exact state); status like `RUNNING / SUCCESS / FAILED` must be strongly consistent and unique per run.
- The "flexibility" MongoDB offers is already covered by **JSONB** for node config while the identity relations stay strictly typed.

When MongoDB is the right answer — be honest

- Documents are genuinely heterogeneous and evolve fast; you can't define stable relational tables.
- You need sharding / write-anywhere horizontal scale from day one.
- The consistency model can be relaxed (e.g. activity logs, event streams, content feeds).

5. Why This Map Fits Nodebase

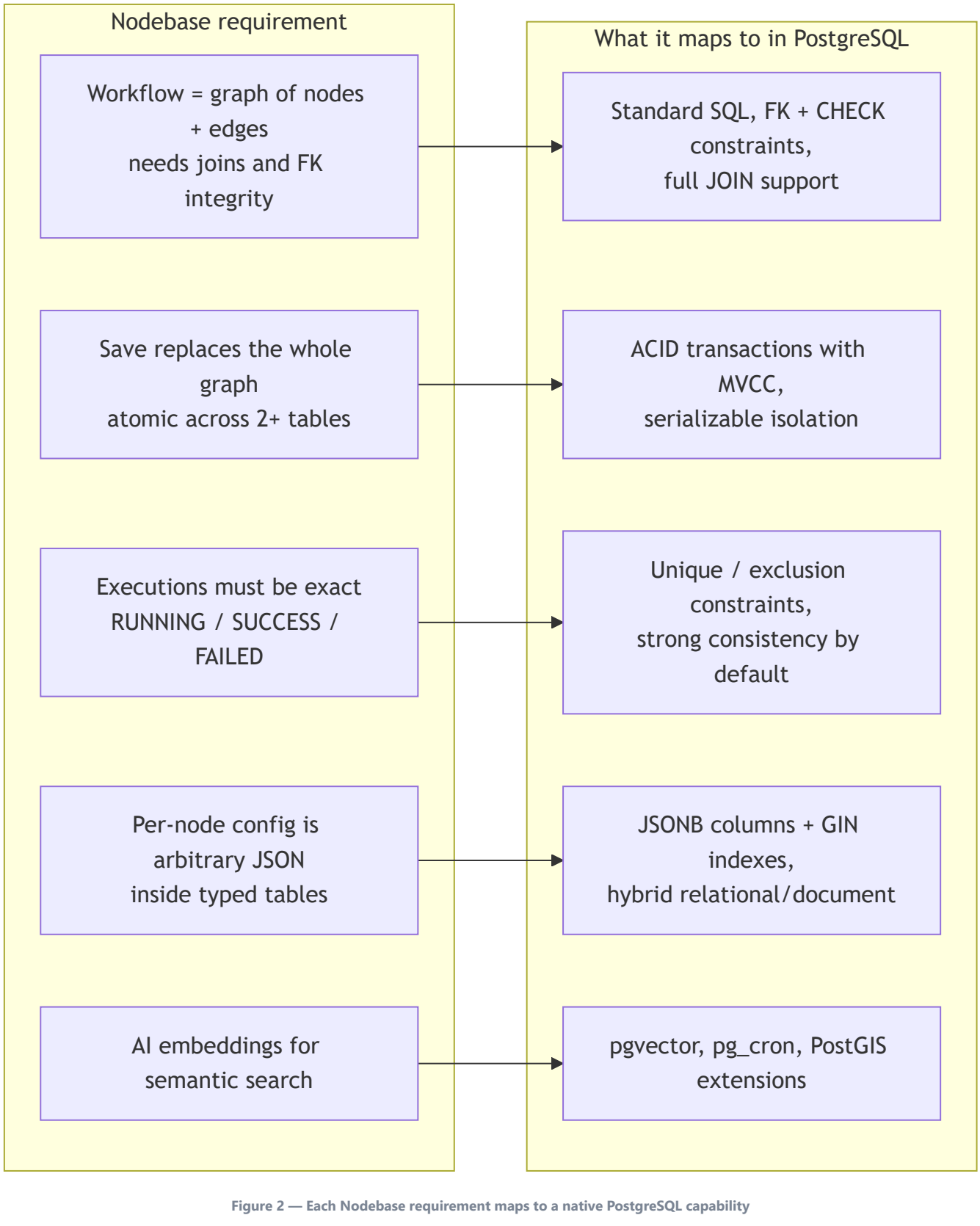


Figure 2 — Each Nodebase requirement maps to a native PostgreSQL capability

Backed by a decision tree

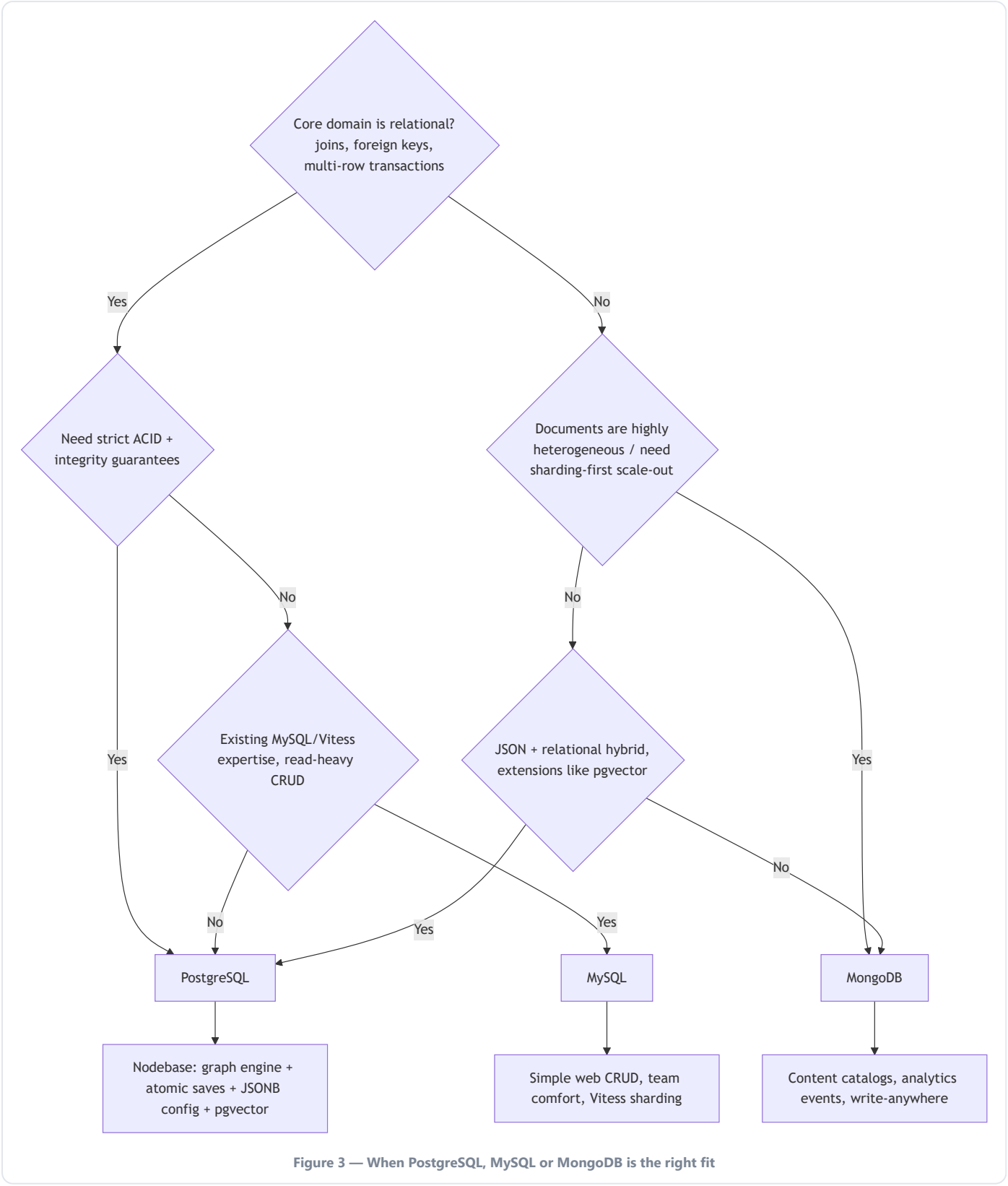


Figure 3 — When PostgreSQL, MySQL or MongoDB is the right fit

The decision is not "Postgres is always better" but "which engine enforces the integrity our domain needs while still covering the flexible/AI parts". For a graph-based workflow product the answer is PostgreSQL.

6. Supporting Examples from a Real Codebase

The claims above are better said with proof. These are drawn from Nodebase (Prisma + Postgres).

ER diagram of the product domain (see Section 7 for the full view):

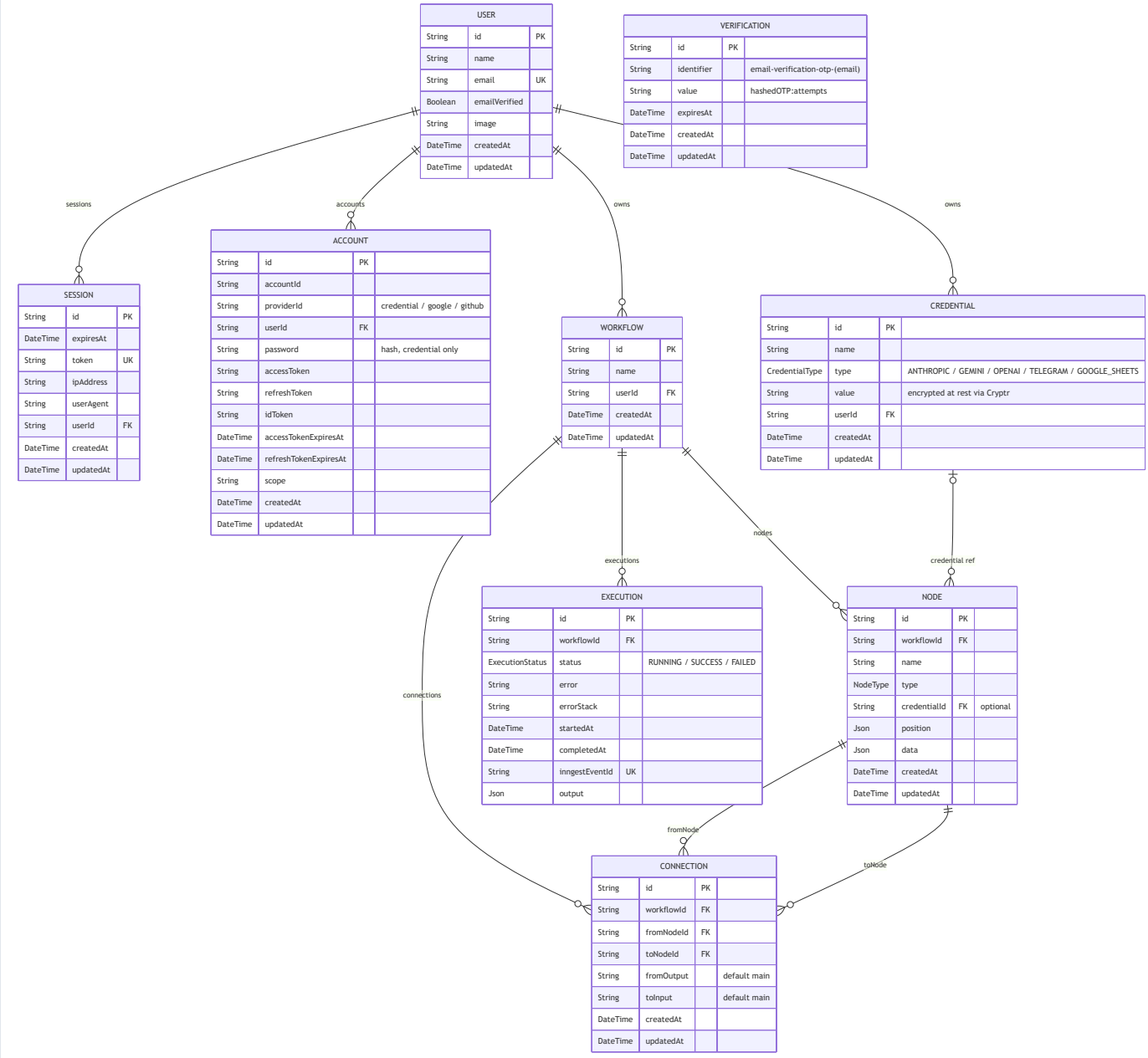


Figure 4 — Nodebase relational model: users own workflows, nodes & connections form the graph, executions record runs

6.1 Relational modeling: the workflow graph

Workflow runs are dispatched to an execution engine that needs the *edges*; a classic join problem a document store makes you re-implement by hand.

```
// Nodebase inngest/utils.ts - load graph, then topologically sort the edges
const edges = await prisma.connection.findMany({
  where: { workflowId },
  select: { fromNodeId: true, toNodeId: true },
});
const sorted = toposort(edges.map(e => [e.fromNodeId, e.toNodeId]));
// toposort throws on a cycle - a guarantee MongoDB cannot give you
```

In MongoDB this becomes a manual graph traversal with `$lookup` pipelines and no integrity that edges point at real nodes; in Postgres the FK gets checked by the engine on every write.

6.2 Atomic multi-table save (why ACID matters)

Saving the editor snapshot deletes and recreates every node and edge in one transaction. If the process dies halfway, MySQL/Postgres both roll back; but the guarantee that two *related* tables switch together is the point for users.

```
// Nodebase workflows.update - single transaction, no partial saves
await prisma.$transaction(async (tx) => {
  await tx.connection.deleteMany({ where: { workflowId } });
  await tx.node.deleteMany({ where: { workflowId } });
  await tx.node.createMany({ data: nodes });
  await tx.connection.createMany({ data: edges });
});
```

6.3 Constraints that prevent bugs

A unique index ties every `Execution` row to one Inngest run — retries and webhook dupes cannot produce two execution rows; an FK guards credential deletion. These are the guards a schemaless store doesn't enforce.

```
-- unique link: one Execution row per Inngest run
CREATE UNIQUE INDEX "execution_inngestEventId_key" ON "Execution"("inngestEventId");

-- FK: a node cannot reference a deleted credential (ON DELETE SET NULL)
ALTER TABLE "Node" ADD CONSTRAINT "Node_credentialId_fkey"
  FOREIGN KEY ("credentialId") REFERENCES "Credential"(id) ON DELETE SET NULL;
```

6.4 JSONB: document flexibility inside a relational schema

Prisma `Json` maps to `jsonb`, so per-node configuration and canvas position stay arbitrary JSON — exactly what MongoDB would offer — yet the row still belongs to a typed, FK-ed table and can be found with a GIN index.

```
-- Node.data is JSONB; find every HTTP_REQUEST node configured for POST
SELECT "id", "name" FROM "Node"
WHERE "type" = 'HTTP_REQUEST'
  AND "data" @> '{"method": "POST"}';   -- @> is the JSONB "contains" operator
```

6.5 Analytics: window functions over executions

```
-- latest execution per workflow with ROW_NUMBER (window function)
WITH ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY "workflowId" ORDER BY "startedAt" DESC) AS rn
  FROM "Execution"
)
SELECT * FROM ranked WHERE rn = 1;
```

6.6 Forward-looking: pgvector for AI

An AI-workflow SaaS will want semantic search over node outputs. pgvector adds cosine/similarity operators and indexes in a schema migration — no separate vector database, no second system to operate. This is the kind of extensibility that keeps Postgres relevant years after the initial schema was written.

7. A Practical Script to Practice

Version A — 30-second answer

"PostgreSQL is an open-source, ACID-compliant relational database. We picked it over MySQL because our domain is genuinely relational — graph nodes, edges and execution lineage — so we need enforced foreign keys, transactions that span multiple tables, and stricter integrity than MySQL's permissive modes. We picked it over MongoDB because the 'flexible schema' benefit is covered by JSONB, while Multi-document transactions, joins and constraints are first-class only in the relational engine."

Version B — 2-minute answer (problem-driven, C2D style)

Context: Nodebase is an AI-workflow SaaS; users glue nodes (triggers/actions) into a graph and run them.
Problem: the model is relational by nature — nodes → edges → executions — and a save must be atomic across tables. A document-first store forces us to re-build joins, constraints and transactions by hand.
Decision: PostgreSQL. Enforced FKs stop orphaned edges; `$transaction` makes graph saves atomic; unique indexes guarantee one execution per run; JSONB preserves per-node config flexibility; pgvector covers AI embeddings later.
Comparison: MySQL 8 has improved, but sits a step behind on JSONB, GIN/GiST indexes, extensions and strictness; MongoDB shines for heterogeneous documents and sharding, which isn't our shape.
Trade-off: Postgres has more operational surface (vacuum, WAL tuning); we accept that for correctness. If the product were a read-mostly CRUD service we'd happily run MySQL; if it were a content catalog we'd use MongoDB.

Rehearse the one-liners

Interviewer asks	Hook line you can open with
"Why not MongoDB"	"MongoDB is great when the document is the unit of work; ours isn't — the graph and its lineage are."
"Why not MySQL"	"MySQL is fine for read-heavy CRUD; we need multi-table atomic writes and advanced query features, where Postgres is more complete."
"What do you gain"	"One engine that is relational when I need integrity, JSON when I need flexibility, and extensible for AI."
"What do you lose"	"Operational simplicity and trivial sharding compared with MySQL or Mongo — we trade that for correctness."

8. Cheat-Sheet Takeaway

Scenario	Best fit	Why
Workflow/graph engine, billing, orders, anything where atomic multi-record writes + integrity are critical	PostgreSQL	ACID, FKs, constraints, JSONB hybrid, extensions
Simple read-heavy web/dashboard CRUD, established MySQL shop	MySQL	Small surface, huge talent pool, mature replication
Heterogeneous documents, sharding-first scale-out, relaxed consistency OK	MongoDB	Schemaless, native horizontal scale
JSON + relational + AI embeddings in one system	PostgreSQL	JSONB + pgvector — no second database

Memorize the **narrative**, not the bullets: "relational domain → need integrity → Postgres provides it and still covers flexibility with JSONB and AI with pgvector. MySQL is less complete here, MongoDB rests the wrong trade-off."